

Day 16 - Notes

DEBUGGING, DEVELOPER TOOLS, PERFORMANCE & MAINTAINABILITY (NOTES)

Debugging in Enterprise Systems

Debugging means finding and fixing errors in a system by analyzing logs, code behavior, and system execution flow.

In Salesforce, debugging becomes complex because many things run at the same time:

- Triggers
- Flows
- Batch jobs
- Integrations
- User actions

Why debugging is difficult

- Multiple automations run together
 - Data flows through many layers
 - Async processes (queue, batch, future methods)
 - High number of users at same time
-

Developer Tools in Salesforce

1. Developer Console

Used for:

- Running SOQL queries
- Viewing logs
- Executing Apex code

- Checking errors
- Helps in quick debugging and testing.
-

2. Debug Logs

Logs show:

- What code executed
 - Order of execution
 - Errors and exceptions
 - Flow and trigger behavior
- Main tool for finding root cause of issues.
-

3. Apex Replay Debugger

Used to:

- Replay execution step-by-step
 - Find exact failure line
 - Check variable values during execution
 - Understand logic flow clearly
- Very powerful for complex Apex debugging.
-

LWC Best Practices

Performance

- Avoid unnecessary re-rendering
 - Reduce server calls
 - Use efficient data handling
 - Keep components lightweight
-

Reusability

- Break UI into small components
 - Reuse common logic
 - Avoid duplicate code
-

Clean Architecture

- Separate UI and logic
 - Use service layers for API calls
 - Keep code structured and readable
-

Maintainability

Good code should be:

- Easy to debug
 - Easy to update
 - Easy to extend
 - Easy to understand
-

1. Bug Analysis

Duplicate Notifications

Possible causes:

- Trigger running twice
- Flow + Apex both sending notification
- Missing condition check

Fix approach:

- Check debug logs
 - Add condition to prevent duplicates
 - Review automation order
-

Wrong Attendance Calculation

Possible causes:

- Incorrect formula logic
- Wrong data input
- Date/time mismatch

Fix approach:

- Verify calculation logic
 - Check input data
 - Compare expected vs actual values
-

Flow Not Triggering

Possible causes:

- Entry condition not satisfied
- Wrong object/event type
- Permission issue

Fix approach:

- Check flow conditions
 - Review debug logs
 - Verify record updates
-

Approval Process Stuck

Possible causes:

- Missing approver
- Incorrect approval criteria
- Record locked

Fix approach:

- Check approval steps
 - Validate user access
 - Inspect approval queue
-

2. Performance Thinking (50,000 Users System)

UI Problems

- Slow page load
 - Heavy components
 - Too many re-renders
-

Backend Problems

- Slow Apex execution
 - Governor limits hit
 - Inefficient queries
-

Database Problems

- Large unoptimized queries
 - Missing indexing
 - Duplicate records
-

Notification Problems

- Duplicate emails
 - Delayed messages
 - Queue overload
-

Automation Problems

- Infinite trigger loops
 - Flow recursion
 - Batch job delays
-

3. Maintainability Thinking

Why modular code?

- Easy to debug
 - Easy to reuse
 - Easy to update
-

Why reusable components?

- Reduces duplication
 - Saves development time
 - Keeps system clean
-

Why avoid quick hacks?

- Creates technical debt
 - Hard to debug later
 - Breaks other features
-

Why maintainable systems matter

Because enterprise systems:

- Grow continuously

- Have many users
 - Need frequent changes
 - Must stay stable
-

Reflection

Debugging is not just fixing errors — it is understanding how the system behaves.

In real enterprise systems, a single issue can involve:

UI + backend + database + automation together.

So developers must think logically and trace problems step-by-step instead of guessing.

Good debugging skills help in:

- Faster issue resolution
 - Better system design
 - Fewer production failures
-

Key Learnings

- Debug logs help trace execution flow
 - Developer Console helps test and inspect code
 - Replay Debugger helps analyze step-by-step execution
 - LWC must be optimized for performance
 - Systems must be modular and reusable
 - Enterprise systems fail due to scaling issues
 - Debugging is a core engineering skill
-

Final Understanding

Enterprise development is not just writing code.

It is about:

- Writing clean systems

- Debugging efficiently
- Thinking about scale
- Maintaining performance
- Building long-term stable architecture